Codebase Guide

NeonTreeEvaluation Working Group

A walkthrough of every file and the problem it solves

Contents

1 Overview **1**

2 `main.py` **2**

3 `src/const.py` **2**

4 `src/utils/download.py` **3**
4.1 Streaming download with a checksum . . . 3
4.2 Recursive unzipping . . . 3
4.3 Fixing double-nested directories . . . 4

5 `src/utils/singleton.py` **4**

6 `src/utils/logger.py` **5**

7 `src/utils/transforms.py` **6**

8 `src/utils/normalization_params.py` **6**

9 `src/data/setup.py` **7**
9.1 Why preprocess and cache? . . . 7
9.2 The preprocessing loop . . . 7
9.3 Merging RGB and CHM . . . 8
9.4 Parsing bounding box annotations . . . 8

10 `src/dataset.py` **9**

11 How the Files Fit Together **10**

Overview

The code in this repository does one thing right now: take the raw NeonTreeEvaluation dataset as downloaded from the internet and transform it into a form that a PyTorch model can consume directly. None of the model architecture or training logic is written yet. What exists is the data pipeline – the plumbing that sits between the raw files on disk and the training loop.

The pipeline has to solve several concrete problems that are not obvious until you actually work with the dataset:

- The download is large and may fail partway through; we need to detect this and avoid silently training on corrupted data.
- The archive contains zip files inside zip files; a single unzip is not enough.
- After unzipping, some directories are double-nested in a way that breaks any code that assumes a flat structure.
- The raw images are GeoTIFF files. Converting them to PyTorch tensors takes several seconds per image. Doing this conversion on every training run would be prohibitively

slow, so we do it once and save the result.
- The RGB imagery and the LiDAR canopy height (CHM) imagery were captured at different resolutions, so they do not line up pixel-for-pixel out of the box.
- The annotation files list bounding boxes in XML format. When a file contains only a single tree, the XML parser returns a different data structure than when it contains multiple trees, which would silently drop single-tree annotations if not handled.

The sections below walk through each source file in the order you would encounter it if you traced a run from `main.py` downward.

main.py

main.py

The entry point. Run this file to execute the full setup pipeline.

```
from src.utils.download import download_data, cleanup_files
from src.const import DATA_PATH, NEON_TREE_PATH
from src.data.setup import SetupNeonTreeData
from pathlib import Path

def main():
    if not Path(DATA_PATH).exists():
        download_data(DATA_PATH)
    cleanup_files(NEON_TREE_PATH)
    data = SetupNeonTreeData(force_overwrite=True)
    print("Data loaded successfully.")
```

`main.py` is deliberately thin. It sequences three steps:

1. Download the raw dataset if the local `data/` directory does not yet exist.
2. Fix the directory structure problems left by the unzipper (see `download.py`).
3. Run the full preprocessing pipeline to convert raw images into cached PyTorch files.

Having a single entry point like this means anyone on the team can set up the project from scratch by running one command, without needing to know the internal details of how the data is obtained or organised.

src/const.py

src/const.py

A single place where every project-wide constant lives.

```
from pathlib import Path
from src.utils.logger import Logger

DATA_URL     = "https://owncloud.gwdg.de/..."
DATA_PATH    = Path("data/")
DATA_HASH    = "0ad723302caa8f34..."
NEON_TREE_PATH = DATA_PATH / "neon_tree" / "NeonTreeEvaluation"
PT_DATA_PATH   = DATA_PATH / "pt_data"
LOG_DIR      = Path("logs/")
LOGGER       = Logger("neon_tree")
```

```
# placeholder values until we compute the real ones from our data
CHANNEL_MEANS  = (0.485, 0.456, 0.406)
CHANNEL_STDEVS = (0.229, 0.224, 0.225)
```

The purpose of `const.py` is to prevent path strings from being scattered across the codebase. If the dataset ever moves to a different directory, or we switch hosting providers for the download, there is exactly one file to edit. Every other file imports its paths from here rather than hardcoding them.

`CHANNEL_MEANS` and `CHANNEL_STDEVS` are the normalisation statistics used to preprocess images before they enter the model. The values currently in the file are the ImageNet statistics (a standard starting point from the ML primer), marked with a comment indicating they are placeholders. The `normalization_params.py` utility exists to compute our dataset-specific values, at which point these constants will be updated.

`src/utils/download.py`

`src/utils/download.py`

Downloads the dataset, verifies its integrity, and unpacks nested archives.

Streaming download with a checksum

The dataset file is large. A naive download that reads the entire response into memory would crash on any machine without enough RAM, and a download that saves to disk without verification could silently produce a corrupted file if the connection dropped partway through.

`download_data` solves both problems at once by streaming the response in 1 MB chunks, writing each chunk to disk and feeding it into a SHA-256 hash function simultaneously:

```
h = hashlib.sha256()
for chunk in response.iter_content(chunk_size=1 << 20):
    f.write(chunk)
    h.update(chunk)

digest = h.hexdigest()
if digest != DATA_HASH:
    temp_file.unlink()          # delete the corrupted file
    raise ValueError(...)       # and refuse to continue
```

SHA-256 is a mathematical function that produces a fixed-length fingerprint from any input. The fingerprint stored in `DATA_HASH` was computed from the original, known-good archive. If any byte in the downloaded file differs – due to a dropped connection, a server error, or file corruption – the fingerprint will not match and the code raises an error before any further processing happens. The incomplete file is deleted automatically so a second attempt starts fresh rather than skipping the download because a partial file already exists.

The `rich` library's `Progress` class provides the animated download progress bar. This is purely cosmetic and can be disabled by passing `verbose=False`.

Recursive unzipping

The NeonTreeEvaluation archive is a zip file that contains further zip files inside it (one per data modality). A single `unzip` call extracts the outer archive but leaves the inner ones untouched.

`_unzip_recursive` handles this by extracting an archive and then scanning the result for any further `.zip` files, repeating until none remain:

```
def _unzip_recursive(zip_path, dest, verbose=True):
    with ZipFile(zip_path, "r") as zf:
        zf.extractall(dest)

    for extracted in dest.rglob("*.zip"):
        if extracted == zip_path:
            continue
        _unzip_recursive(extracted, extracted.parent / extracted.stem,
            verbose)
        extracted.unlink()    # delete the .zip once extracted
```

Each inner archive is extracted into a subdirectory named after the archive itself (e.g. `RGB.zip` extracts into `RGB/`), and then the `.zip` is deleted to save disk space.

Fixing double-nested directories

After extraction, the dataset has a structural quirk: the `evaluation/` and `annotations/` directories each contain a subdirectory with the same name as themselves. That is, the files end up at paths like:

`NeonTreeEvaluation/evaluation/evaluation/SJER_006.tif`

instead of the expected:

`NeonTreeEvaluation/evaluation/SJER_006.tif`

`cleanup_files` detects this pattern and collapses it by moving every item from the inner directory up to the outer one:

```
def remove_nested(save_path, target_name):
    dir        = save_path / target_name
    nested_dir = dir / target_name          # e.g. evaluation/evaluation
    if nested_dir.exists():
        for item in nested_dir.iterdir():
            shutil.move(str(item), dir / item.name)
        nested_dir.rmdir()

remove_nested(save_path, "evaluation")
remove_nested(save_path, "annotations")
```

Without this step, every file-loading function would need to know about the double nesting and construct paths accordingly. Fixing it once at setup time means the rest of the code can assume the straightforward structure.

`src/utils/singleton.py`

`src/utils/singleton.py`

A metaclass that ensures a class can only ever be instantiated once.

```
class SingletonMeta(type):
    _instances = {}
```

```
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            instance = super().__call__(*args, **kwargs)
            cls._instances[cls] = instance
        return cls._instances[cls]
```

A *metaclass* is a class whose instances are themselves classes. That is a confusing sentence, so here is the practical effect: any class that uses `SingletonMeta` as its metaclass will, the second time you try to create an instance, just hand you back the instance that was created the first time.

Why does the code do this?

`SetupNeonTreeData` (described in Section 9) is a singleton because the preprocessing work it does – reading every image from disk and saving it as a `.pt` file – is expensive and should only run once per execution. If multiple parts of the codebase each tried to construct a `SetupNeonTreeData` object independently, without the singleton guarantee, they would each kick off the full preprocessing run again. The singleton ensures that only the first construction does real work; subsequent constructions silently return the already-initialised object.

src/utils/logger.py

src/utils/logger.py

A logging wrapper that writes structured records to a file and pretty output to the terminal simultaneously.

```
class Logger:
    def __init__(self, name):
        self.console = Console()          # rich terminal output
        self._logger = self._setup_logger(name)

    def log_and_print(self, rich_object):
        # render the object to plain text and write to the log file
        ...
        self._logger.info(plain_text)
        # also render it to the terminal with colour and formatting
        self.console.print(rich_object)

    def __getattr__(self, name):
        return getattr(self._logger, name)    # delegate .debug(),
            .warning() etc.
```

Python's built-in `logging` module writes plain text to log files. The `rich` library renders coloured, formatted output to the terminal. `Logger` combines both: `log_and_print` sends output to both destinations at once, while the `__getattr__` delegation means you can also call `LOGGER.debug(...)`, `LOGGER.warning(...)` and so on exactly as you would with a standard Python logger (these go to the file only, without terminal output).

Each run of the program creates a new log file named with the current timestamp (e.g. `logs/2026-03-18_14-32-07.log`). This means you can look back at the output from previous runs without it being overwritten.

A single shared `LOGGER` instance is created in `const.py` and imported wherever it is needed, which keeps all log records for a given run in one file.

src/utils/transforms.py

src/utils/transforms.py

A custom Torchvision transform that converts a raw image array into a normalised float tensor.

```
class ToTensor(torch.nn.Module):
    def forward(self, img, label=None):
        converted = v2.Compose([
            v2.ToImage(),
            v2.ToDtype(torch.float32, scale=True),
        ])(img)

        if label is not None:
            return converted, label
        else:
            return converted
```

Two conversions happen here in sequence:

1. `v2.ToImage()` converts the NumPy array returned by the image reader into a PyTorch tensor and reorders the dimensions from `(H, W, C)` to `(C, H, W)` – PyTorch's expected convention.
2. `v2.ToDtype(torch.float32, scale=True)` changes the numeric type from unsigned 8-bit integers (values 0–255) to 32-bit floats (values 0.0–1.0). The `scale=True` argument performs the division by 255 automatically.

Why does the code do this?

Why divide by 255? Neural network optimisers (the algorithms that update the model's weights during training) are sensitive to the scale of the numbers they see. Input values of 0–255 and weight values of approximately 0.0–1.0 are on very different scales, which makes training unstable. Rescaling the inputs to 0.0–1.0 keeps everything in a compatible range.

The transform accepts an optional `label` argument and passes it through unchanged. This allows it to be used in contexts where an image and its annotation are processed together without accidentally discarding the annotation.

src/utils/normalization_params.py

src/utils/normalization_params.py

A utility script for computing the channel-wise mean and standard deviation of the dataset, to be run once and used to update `CHANNEL_MEANS` and `CHANNEL_STDEVS` in `const.py`.

```
def get_images(data_dir):
    files     = sorted(data_dir.glob("*.pt"))
    unpickles = [torch.load(file, weights_only=False) for file in files]
    images    = [np.asarray(image) for image, boxes in unpickles]
    return np.stack(images)
```

This file is a standalone script (note the `if __name__ == "__main__"` block at the bottom) rather than a module imported by the rest of the code. Its job is to iterate over the cached `.pt` files, load the image tensors, stack them into a single large array, and compute statistics across the whole training set.

Note

This script is not yet complete. The `means()` function currently computes the per-channel means but does not return them – a missing `return` statement – and there is no corresponding function for the standard deviations. Once these are implemented, the values should replace the ImageNet placeholders in `const.py`.
The reason we compute these from our own data rather than reusing ImageNet values permanently is that our images are aerial photographs, which have a different brightness distribution than ground-level photographs. Dataset-specific normalisation values will give the model a more consistent input distribution to work with.

`src/data/setup.py`

`src/data/setup.py`

The main preprocessing class. Reads raw TIFFs and XML annotations, combines RGB and CHM channels, and saves everything to disk as `.pt` files for fast loading during training.

Why preprocess and cache?

Loading a raw GeoTIFF, resizing it, converting it to a tensor, and parsing an XML file takes several seconds per image. A training run with thousands of iterations, each loading a new image, would spend more time on disk I/O than on actual learning. The standard solution is to do all the slow preprocessing once and save the result in a format that is fast to load – in our case, PyTorch's native `.pt` format.

`SetupNeonTreeData` uses the singleton pattern from `singleton.py` to guarantee that this preprocessing only ever runs once per program execution, regardless of how many times different parts of the code ask for a `SetupNeonTreeData` object.

The preprocessing loop

For each RGB image in the training or evaluation split, the code:

1. Finds the corresponding annotation file (in `annotations/`) and CHM file (in `CHM/`) by constructing the expected paths from the image filename.
2. Skips the image with a warning if either the annotation or the CHM is missing – a graceful fallback rather than a crash.
3. Loads the RGB image as a tensor of shape `(3, H, W)`.
4. Loads the CHM image and resizes it to match the RGB spatial dimensions, since the two modalities were captured at different resolutions.
5. Concatenates RGB and CHM along the channel dimension, producing a 4-channel tensor of shape `(4, H, W)`.

6. Parses the bounding boxes from the XML annotation file.
7. Saves the `(image_tensor, bounding_boxes)` pair to a `.pt` file.

Merging RGB and CHM

The channel concatenation step is the most consequential data design decision in the codebase so far:

```
rgb  = self._load_image(img_path)                    # shape: (3, H, W)
chm  = self._load_image(chm_path, target_size=rgb.shape[1:])  # shape:
    (1, H, W)
comb = torch.cat([rgb, chm], dim=0)                  # shape: (4, H, W)
```

By stacking LiDAR height as a fourth channel alongside the three colour channels, the model can learn to use height information alongside colour when deciding whether a blob in the image is a tree crown. This is one of the main reasons to use the NEON dataset over ordinary aerial photography – the height channel makes it much easier to distinguish tall trees from low shrubs that might look similar in colour alone.

Parsing bounding box annotations

The annotation files are in Pascal VOC XML format, the same format used in many standard computer vision datasets. A typical file looks like:

```
<annotation>
  <object>
    <bndbox>
      <xmin>45</xmin> <ymin>112</ymin>
      <xmax>130</xmax> <ymax>198</ymax>
    </bndbox>
  </object>
  <object> ... </object>
</annotation>
```

The `xmltodict` library parses this into a Python dictionary. There is a subtle issue: when the file contains multiple `<object>` entries, `xmltodict` puts them in a list. When there is exactly one entry, it puts it in a dictionary directly (not inside a list). The code handles both cases explicitly:

```
if isinstance(data["annotation"]["object"], list):
    bounding_boxes = [[...] for object_dict in
        data["annotation"]["object"]]
elif isinstance(data["annotation"]["object"], dict):
    bounding_boxes = [[...]]    # wrap the single dict in a list
```

Without this check, images containing only one tree would silently fail to produce any annotations, corrupting the training data for those samples.

The extracted coordinates are wrapped in a `BoundingBoxes` object from Torchvision, which stores not just the coordinates but also the format (`"XYXY"` – two corner points, as opposed to centre-plus-size) and the canvas size (the image dimensions). Torchvision's transform pipeline uses this metadata to update bounding box coordinates correctly when the image is cropped or flipped during augmentation.

src/dataset.py

src/dataset.py

The PyTorch `Dataset` class that the training loop will consume.

```
class TreeImageDataset(Dataset):
    def __init__(self, input_dir, transforms=None):
        SetupNeonTreeData()    # ensure preprocessing has been run
        self.paths = sorted(input_dir.glob("*.pt"))
        self.transform = Compose(transforms) if transforms else
            Compose([
             RandomCrop(400),
             Normalize(CHANNEL_MEANS, CHANNEL_STDEVS),
             RandomHorizontalFlip(),
             RandomVerticalFlip(),
             SanitizeBoundingBoxes(),
        ])

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        file, boxes = torch.load(self.paths[idx])
        file = self.transform(file)
        return file, boxes
```

`TreeImageDataset` is the class that the `DataLoader` described in the ML primer will wrap. It satisfies PyTorch's Dataset contract by implementing `__len__` (how many examples?) and `__getitem__` (give me example number `idx`).

Loading from `.pt` files means `__getitem__` is fast: a single `torch.load` call, then the transform pipeline. No image decoding, no XML parsing, no resizing – all of that was done once during setup.

The default transform pipeline does the following to each example before returning it:

1. **RandomCrop(400)** – cuts a random 400×400 pixel window from the image. This is data augmentation: by showing the model different crops of the same tile each time, we effectively multiply the size of the training set without needing more labelled data.
2. **Normalize** – subtracts the channel means and divides by the standard deviations, shifting the pixel value distribution to be centred near zero. This is expected by the pre-trained model architecture.
3. **RandomHorizontalFlip / RandomVerticalFlip** – randomly mirrors the image left-right and/or top-bottom. Aerial imagery has no canonical orientation (north can point in any direction), so all four orientations are equally valid and showing the model all of them improves robustness.
4. **SanitizeBoundingBoxes** – removes any bounding boxes that ended up fully outside the cropped region after `RandomCrop`, and clamps any partially-outside boxes to the image edge. Without this step, the model would receive annotations pointing to regions that do not exist in the cropped image.

Note that `SanitizeBoundingBoxes` only works correctly because the bounding boxes are stored as a Torchvision `BoundingBoxes` object (rather than a plain tensor). Torchvision's transforms know to treat `BoundingBoxes` specially, applying the same geometric transformation to the boxes

as to the image. If the boxes were stored as a plain `torch.Tensor`, the `RandomCrop` would move the pixels but leave the box coordinates unchanged – a silent, hard-to-detect bug.

Note

The constructor calls `SetupNeonTreeData()` without arguments at startup. Because `SetupNeonTreeData` is a singleton, this either triggers preprocessing (if it has not been done yet) or returns the existing instance immediately (if it has). This means you can construct a `TreeImageDataset` without having run `main.py` first; the dataset will set itself up automatically.

How the Files Fit Together

File	Role	Key output
`main.py`	entry point	runs the full setup
`const.py`	constants	paths, hash, logger, stats
`utils/download.py`	fetch data	verified, flat directory tree
`utils/singleton.py`	pattern	one-instance guarantee
`utils/logger.py`	observability	file + terminal logging
`utils/transforms.py`	conversion	float tensor from raw array
`utils/normalization_params.py`	statistics	dataset mean/stdev (WIP)
`data/setup.py`	preprocessing	`.pt` files on disk
`dataset.py`	training interface	batches of (image, boxes)

The flow is linear: `main.py` calls `download.py` to fetch and flatten the raw data, then calls `setup.py` to convert it into `.pt` files. From that point on, `dataset.py` is the only file that the training loop needs to interact with – everything upstream of it has already done its work.